AI SUPERVISOR ASSISTANT

System Design Document

Architecture & Deep Dives

Requirements | Core Entities | API Design | Data Flow | Deep Dives

PLATFORM	DATABASE	AI MODEL
Vercel + Render	Supabase PostgreSQL	Groq llama-3.3-70b

1. Requirements

1.1 Functional Requirements

#	Requirement	Role
FR-01	Users register with email/password or Google OAuth and choose a role (Manager or Team Member)	Both
FR-02	Managers create projects by entering a name/description or uploading a PDF spec	Manager
FR-03	AI automatically generates tasks and staffing roles from project description or PDF	Manager
FR-04	AI assigns role_id to tasks, matching tasks to the most appropriate role	Manager
FR-05	Candidates (team members) are scored by skill match % per role and displayed sorted highest first	Manager
FR-06	Managers hire/fire team members into specific roles; hired candidates receive email	Manager
FR-07	All users can update task status: Not Started, In Progress, Delayed, Complete	Both
FR-08	Team members can only update tasks belonging to the role they were hired into	Team
FR-09	Managers and role-eligible team members can edit task What and How fields inline	Both
FR-10	Calendar supports creating meetings (with time slots, participants) and deadlines (linked to tasks)	Both
FR-11	Email alerts sent on task updates and project changes to members with alerts enabled	Both
FR-12	Weekly project summary email sent every Monday at 6AM in the user's local timezone	Both
FR-13	AI project chat answers questions and auto-updates task status from natural language	Both
FR-14	Team chat supports direct messages and group channels per project	Both
FR-15	App supports English, Spanish, French, and German with instant language switching	Both
FR-16	Language and date format preferences saved to database and restored on login	Both

1.2 Non-Functional Requirements

Category	Requirement
Performance	API responses under 500ms for standard CRUD; AI generation under 10s for typical projects
Scalability	Stateless Express backend — horizontal scaling via Render; Supabase handles DB scaling
Security	JWT auth on all protected endpoints; Supabase RLS prevents cross-user data access; service role key never exposed to frontend

Availability	Vercel 99.9% SLA for frontend; Render free tier spins down after 15min inactivity (upgrade for always-on)
Reliability	Email delivery via Resend with retry logic; AI generation errors caught and returned gracefully
Maintainability	Modular MVC backend; component-based React frontend; i18next for centralized translations
Internationalization	All UI text translated to EN/ES/FR/DE; dates formatted per user preference; preferences synced to DB
Accessibility	Semantic HTML, focus states on interactive elements, color contrast via Tailwind design tokens

1.3 Capacity Estimation

Estimates are based on a small-to-medium team deployment (10-100 users). Supabase free tier supports 500MB database, 1GB file storage, and 50,000 monthly active users. Groq free tier provides 30 requests/minute with llama-3.3-70b. Resend free tier sends up to 3,000 emails/month to verified addresses.

Metric	Estimate	Bottleneck
Concurrent users	50-200	Render free tier (512MB RAM) — upgrade for 200+
Projects per manager	10-50	No hard limit; JSONB roles stored per project row
Tasks per project	5-100	AI generation tested up to 50 tasks; embeddings optional
API requests/day	~10,000	Render free tier handles well within limits
AI calls/day	~500	Groq free: 14,400/day (30/min) — sufficient for typical usage
Emails/month	~1,000	Resend free: 3,000/month — sufficient for small teams
Storage (PDFs + avatars)	~500MB/year	Supabase free: 1GB total — upgrade at ~50 active projects
DB size	~50MB/year	Supabase free: 500MB — sufficient for 1-2 years of data

2. Core Entities

The system revolves around 8 core entities stored in Supabase PostgreSQL. All tables have Row Level Security (RLS) enabled. The backend uses the service_role key to bypass RLS where cross-user queries are required.

Entity	Table	Key Fields	Relationships
User / Profile	profiles	id, role, email_alerts, weekly_reports, preferred_language	Root entity — all others reference profiles
Project	projects	id, owner_id, name, status, roles (jsonb), deadline	owner_id → profiles; has many tasks, members, events
Task	tasks	id, project_id, title, what, how, status, role_id, role_title	project_id → projects; assigned_to → profiles
Project Member	project_members	project_id, user_id, role_id, role_title, hired_at	Links profiles to projects via role; unique(project, user, role)
Upload	uploads	id, project_id, filename, file_url, raw_text	project_id → projects; raw_text fed to AI
Calendar Event	calendar_events	id, type, event_date, event_time, participants (uuid[])	project_id, task_id, created_by → profiles
Chat Message	chat_messages	id, project_id, channel, sender_id, role, content	channel = "ai" or chat_channels.id
Chat Channel	chat_channels	id, project_id, name, members (uuid[]), is_group	project_id → projects; members array of profile ids

Entity State Machines

Task Status Transitions

From	To	Who Can Transition	Trigger
not_started	in_progress	Manager or hired role member	User clicks In Progress in task panel
in_progress	delayed	Manager or hired role member	User clicks Delayed in task panel
in_progress	complete	Manager or hired role member	User clicks Complete in task panel
delayed	in_progress	Manager or hired role member	User resumes work
any	complete	AI via chat	AI detects completion in natural language message
any	any	Manager only	Manager can override any status at any time

3. API & System Interface

The backend exposes a RESTful JSON API at /api/*. All protected routes require Authorization: Bearer in the request header. The token is a Supabase JWT issued on login and stored in localStorage.

Route Group	Base Path	Key Endpoints
Auth	/api/auth	POST /register, POST /login
Users	/api/users	GET /me, GET /profile, PUT /profile, PUT /preferences, DELETE /me
Projects	/api/projects	GET /, GET /:id, POST /, PUT /:id, PUT /:id/roles, DELETE /:id
Tasks	/api/tasks	GET /:projectId, POST /, PATCH /:id/status, PATCH /:id/assign, PUT /:id, DELETE /:id, POST /assign-roles/:projectId
Members	/api/projects	GET /:id/candidates?role_id=, GET /:id/members, POST /:id/hire, DELETE /:id/fire/:userId
Uploads	/api/uploads	POST /:projectId (multipart/form-data PDF upload)
Calendar	/api/calendar	GET /, POST /, DELETE /:id, GET /colleagues
Chat	/api/chat	POST /:projectId (AI), GET /:projectId/history, GET|POST /:projectId/channels, GET|POST /channels/:id/messages, DELETE /messages/:id

Authentication Flow

Step	Action	Details
1	User submits email + password	POST /api/auth/login
2	Backend calls supabase.auth.signInWithPassword()	Returns Supabase session
3	Backend fetches profile from profiles table	Gets role, preferences
4	Returns { user, token } to frontend	Token = Supabase access_token (JWT)
5	Frontend stores token in localStorage	Used in Authorization header
6	protect() middleware calls supabase.auth.getUser(token)	Validates on every request
7	requireAuth middleware fetches profile via supabaseAdmin	Attaches req.user with role

4. Data Flow

4.1 Project Creation with AI

Flow: Create Project
■ Manager submits project name + description (or uploads PDF)
■ Frontend sends POST /api/uploads/:projectId with FormData
■ Backend extracts text from PDF using unpdf library (or uses name + description)
■ AI Step 1: parseProjectRoles(text) → Groq generates role objects with IDs
■ Roles saved to projects.roles (JSONB column)
■ AI Step 2: parseNewProject(text, roles) → Groq generates tasks assigned to role IDs
■ Tasks inserted into tasks table with project_id and role_id
■ Response: { tasks[], roles[] } returned to frontend
■ Frontend shows TasksEditor and RolesEditor for review
■ Manager saves → tasks and roles written to DB
■ Manager navigates to Candidates page to hire

4.2 Candidate Scoring & Hiring

Flow: Hire a Candidate
■ Manager opens Candidates page → selects project → clicks a role
■ Frontend calls POST /tasks/assign-roles/:projectId (AI fixes any null role_ids)
■ Frontend calls GET /projects/:id/candidates?role_id=role-1
■ Backend builds skill pool from ROLE_SKILL_MAP (15 skills per role type keyword)
■ For each team member: match% = (skills in common / total expected skills) x 100
■ Backend returns candidates sorted by match% descending (hired members first)
■ Manager selects candidates and clicks Hire
■ POST /projects/:id/hire → inserts row in project_members
■ Resend sends hiring email to new team member
■ Candidates page refreshes role card showing hired member

4.3 AI Chat with Task Auto-Update

Flow: AI Chat Auto-Update
■ User types message in AI chat tab (e.g. "I finished the login page")
■ Frontend sends POST /api/chat/:projectId with { message, history[] }
■ Backend loads project + all tasks from Supabase for context
■ System prompt built: project name, description, task list (IDs internal only — never shown)

- Message + history sent to Groq llama-3.3-70b via LangChain
- AI responds with natural language + optional JSON block
- Backend strips from response text (user never sees it)
- Backend applies updates to tasks table (status, what, how)
- Both user message and AI response saved to chat_messages table
- Response { text, taskUpdates[] } returned to frontend
- Frontend updates task list in real time without page refresh

4.4 Email Alert Flow

Flow: Email Alert

- Task status updated via PATCH /api/tasks/:id/status
- updateTaskStatus validates role permission (team member must match task role_id)
- Task updated in Supabase
- notifyTaskStatusUpdate() called asynchronously (non-blocking)
- NotificationService queries: project owner + all project_members where email_alerts = true
- alertEmailTemplate() generates styled HTML email
- Resend sends email to each recipient
- Same pattern for: project updates, calendar events (bypass toggle)

4.5 Weekly Report Flow

Flow: Weekly Report

- node-cron runs every hour: 0 * * * *
- For each user with weekly_reports = true:
- Calculate user local time from time_zone offset string
- If local time = Monday 6:00 AM → trigger report
- buildUserReport(): fetch all projects (owned + hired into)
- For each project: count completed this week (updated_at > 7 days ago + status = complete)
- weeklyReportTemplate() generates multi-project HTML summary email
- Resend delivers to user inbox

5. High Level Design

System Architecture

The system follows a classic three-tier architecture: **Presentation layer** (React SPA on Vercel), **Application layer** (Express REST API on Render), and **Data layer** (Supabase PostgreSQL). External services (Groq AI, Resend email, Google OAuth) are integrated at the application layer. The frontend never communicates directly with the database — all data access goes through the API.

BROWSER / CLIENT
React SPA (Vite + Tailwind) Vercel CDN https://ai-supervisor-app.vercel.app

▼ *REST API (JSON + JWT)*

APPLICATION LAYER
Express.js REST API Render Web Service https://ai-supervisor-app.onrender.com Routes: /api/auth, /api/users, /api/projects, /api/tasks, /api/uploads, /api/calendar, /api/chat, /api/projects (members)

▼ *Supabase Client (service role)* ▼ *Groq API* ▼ *Resend API* ▼ *Google AI*

DATA LAYER	AI LAYER	EMAIL	AUTH
Supabase PostgreSQL + Auth + Storage	Groq llama-3.3-70b LangChain	Resend Transactional Email API	Google OAuth + Supabase Auth

Component Architecture

Layer	Components	Responsibility
Pages	Dashboard, AddProject, Candidates, Settings, ProjectView	Top-level route components — fetch data, manage state
Layout	Sidebar, FloatingChat	Persistent navigation and chat bubble
Feature Components	ProjectCard, TaskItem, CandidateCard, DeadlineCalendar, ProjectChat	Self-contained feature UIs
Dashboard Components	StatsCards, DonutChart, ProjectList	Dashboard widgets
Settings Components	Profile, Notifications, Appearance, SettingMenu	Settings sub-pages
Context / State	AuthProvider, AppSettingsProvider, useAuth, useAppSettings	Global state for auth and preferences
i18n	i18n.js, react-i18next useTranslation()	All UI text in 4 languages

6. Deep Dives

6.1 AI Task & Role Generation Pipeline

The AI pipeline uses LangChain with Groq's llama-3.3-70b-versatile model. Generation is split into two sequential steps to ensure tasks are properly assigned to roles at creation time, avoiding the need for post-hoc role assignment.

Step	Function	Prompt Strategy	Output
1	parseProjectRoles(text)	Single prompt: extract staffing roles with IDs, titles, counts, and skills from project description	roles[] with stable IDs (role-1, role-2...)
2	parseNewProject(text, roles)	Two-part prompt: first understand roles, then generate tasks assigned to role IDs	tasks[] each with role_id and role_title
3 (update)	parseProjectUpdate(text, existingTasks, roles)	Merge prompt: compare new requirements against existing tasks, update changed ones, add new ones	change[] with task_id (update) or no id (new)
4 (fix)	assignRolesToTasks(projectId)	Match task titles/skills to available roles	Updates null role_id tasks in DB

6.2 Candidate Scoring Algorithm

Candidates are scored by comparing their profile skills against an expected skill set derived from the role type. The system uses a ROLE_SKILL_MAP with 15 skills per role keyword. Multiple keywords can match (e.g. "Full Stack Developer" matches both "full stack" and "developer"), and matched skills are merged into a deduplicated pool.

Skill Pool Priority (highest to lowest):

1. Tasks explicitly assigned to this role (role_id matches) — most accurate
2. Skills defined on the role object in the project roles JSONB
3. ROLE_SKILL_MAP lookup by role title keywords (15 skills per keyword)
4. All project task skills as fallback (least specific)

```
match% = (candidate skills that appear in skill pool) / (total skills in pool) x 100
```

Hired members always sort first. Available candidates sort by match% descending. The ROLE_SKILL_MAP covers: backend, frontend, fullstack, devops, AI/ML, UX/design, QA, security, project manager, data, mobile.

6.3 Real-Time Chat Architecture

Feature	Implementation	Trade-off
AI Chat persistence	Messages saved to chat_messages with channel="ai" after each exchange	Full history on reload; no WebSocket needed
Team message delivery	3-second polling on GET /channels/:id/messages	Simple and reliable; slight latency vs WebSocket

AI task auto-update	AI response contains JSON block stripped before display; backend applies updates silently	Seamless UX; AI never exposes internal IDs
Group membership	members uuid[] column on chat_channels; listChannels filters by created_by OR members contains user	Both creator and recipients see the channel
Message deletion	DELETE /chat/messages/:id validates sender_id = req.user.id	Users can only delete own messages

6.4 Email Notification System

Component	File	Role
NotificationService	Utils/NotificationService.js	Queries recipients, calls Resend, respects email_alerts toggle
AlertEmailTemplate	Utils/AlertEmailTemplate.js	Generates styled dark-theme HTML for instant alerts
WeeklyReportTemplate	Utils/WeeklyReportEmailTemplate.js	Multi-project summary HTML with per-role stats
WeeklyReportJob	Utils/WeeklyReportJob.js	node-cron hourly check; fires at Monday 6AM per user timezone
calendarEvents controller	Controllers/calendarEvents.controller.js	Calendar emails bypass email_alerts — always sent
taskController	Controllers/taskController.js	Calls notifyTaskStatusUpdate() after status change (non-blocking)
uploadsController	Controllers/uploadsController.js	Calls notifyProjectUpdate() after task merge (updates only)

6.5 Role-Based Access Control

Resource	Manager	Team Member	Enforcement
Add/Edit Project page	Full access	Blocked (redirect to /dashboard)	ManagerRoute.jsx + Sidebar hide
Candidates page	Full access	Blocked (redirect to /dashboard)	ManagerRoute.jsx + Sidebar hide
Team Utilization stat	Visible	Hidden	isManager check in StatsCards.jsx
Task status update	Any task	Only tasks where role_id matches hired role	updateTaskStatus() membership check
Task What/How edit	Any task	Only tasks for their hired role	canEditFields in TaskItem.jsx
Task assignment	Any task, any hired member	Not available	isManager check in TaskItem.jsx
Project deletion	Own projects	Not available	owner_id check in deleteProject()
Calendar event deletion	Own events	Own events	created_by check in deleteEvent()

6.6 Internationalization Architecture

Language switching is handled by i18next + react-i18next. All 4 languages (EN/ES/FR/DE) are bundled in i18n.js at build time — no runtime fetching. Language preference is saved to profiles.preferred_language via PUT /api/users/preferences and loaded on login from the database. AppSettingsProvider polls for a token every second after mount to handle the post-login case. Date format is stored separately as preferred_date_format and applied via a formatDate() helper in context.

Challenge	Solution
Components not re-rendering on language change	key={i18n.language} on root divs forces remount; useTranslation() hook auto-updates
text-white not applying in Tailwind JIT	Inline style={{ color: "#ffffff" }} used for dynamic class strings where JIT cannot detect class name statically
Language not loading on first login	AppSettingsProvider polls localStorage for token every 1s; on detection fetches DB preferences and calls i18n.changeLanguage()
bg-linear-to-r not working (Tailwind v3)	Use bg-gradient-to-r for Tailwind v3; bg-linear-to-r is Tailwind v4 syntax